

SERVER-SIDE RENDERING Z WYKORZYSTANIEM FLASK

SPIS TREŚCI

Spis treści.....	1
Cel zajęć.....	1
Rozpoczęcie.....	1
Uwaga.....	2
Gdzie pracować?.....	2
Środowisko wirtualne	2
Portowanie aplikacji z LAB G/I	3
Commit projektu do GIT.....	15
Podsumowanie.....	15

CEL ZAJĘĆ

Celem głównym zajęć jest zdobycie następujących umiejętności:

- tworzenie i wykorzystanie środowisk wirtualnych Python;
- instalacja i uruchomienie Flask;
- zarządzanie routingami we Flask;
- wykorzystanie szablonów Jinja2;
- portowanie aplikacji webowych CRUD pomiędzy PHP/Express a Flask.

W praktycznym wymiarze uczestnicy wykorzystają prosty framework Flask do odtworzenia funkcjonalności naiwnego frameworka z poprzednich laboratoriów.

ROZPOCZĘCIE

Rozpoczęcie zajęć. Powtórzenie zasad tworzenia i aktywacji środowiska wirtualnego Python. Powtórzenie zasad działania pip. Mechanizm pip freeze. Powtórzenie metod routingu Flask. Powtórzenie szablonów Jinja2.

Wejściówka?

UWAGA

Ten dokument aktywnie wykorzystuje niestandardowe właściwości. Podobnie jak w LAB A wejdź do Plik -> Informacje -> Właściwości -> Właściwości zaawansowane -> Niestandardowe i zaktualizuj pola. Następnie uruchom ten dokument ponownie lub Ctrl+A -> F9.

GDZIE PRACOWAĆ?

Python pozwala na prace w środowisku wirtualnym. W związku z tym wydaje się, że można wrócić do bezpiecznej lokalizacji na dysku I, tj. I:\PTW, a w razie problemów z wydajnością, środowisko wirtualne można przenieść gdzie indziej (dysk C). Nawet w przypadku utraty środowiska wirtualnego, łatwo jest je odtworzyć z requirements.txt.

Na tym etapie warto w ustawieniach tej karty pracy zmienić wartość niestandardowej właściwości labsdir oraz labdir, po czym odświeżyć wszystkie pola w dokumencie Ctrl+A, F9.

ŚRODOWISKO WIRTUALNE

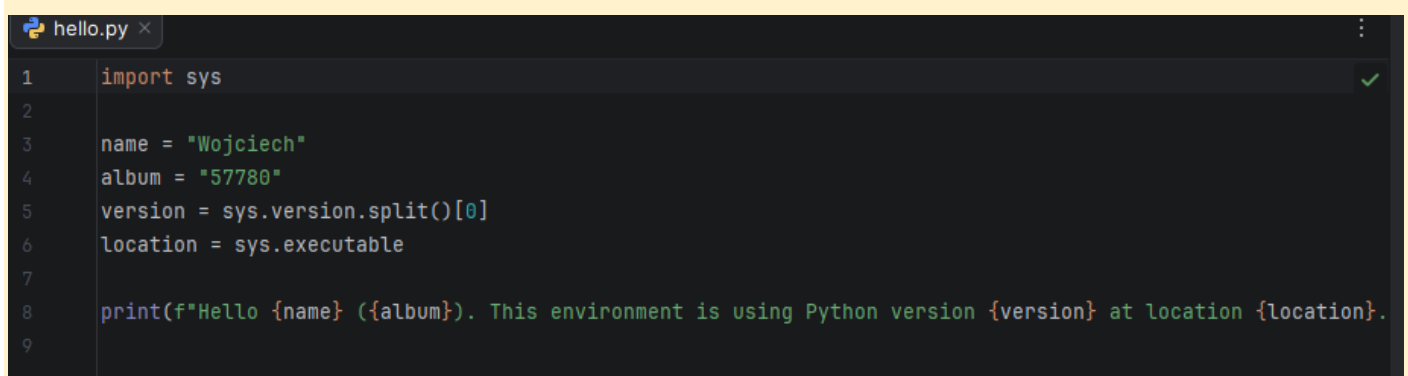
Utwórz katalog na kod tego laboratorium: I:\PTW\lab-j. Otwórz go w PyCharm (nie PhpStorm!). Wykorzystaj PyCharm do utworzenia środowiska wirtualnego. Utwórz plik I:\PTW\lab-j\hello.py i zaimplementuj skrypt, który wygeneruje następujący napis na wyjściu:

```
Hello Wojciech (57780). This environment is using Python version 3.14.0 at location C:\Users\...\venv\Scripts\python.exe.
```

Wykorzystaj f-string, w którym elementy zaznaczone na żółto będą interpolowane do faktycznych wartości.

Otwórz terminal. Aktywuj wirtualne środowisko i uruchom skrypt hello.py. Deaktywuj wirtualne środowisko.

Wstaw zrzut ekranu całości kodu skryptu zgodnie z wymaganiem powyżej.



```
1 import sys
2
3 name = "Wojciech"
4 album = "57780"
5 version = sys.version.split()[0]
6 location = sys.executable
7
8 print(f"Hello {name} ({album}). This environment is using Python version {version} at location {location}.
```

Wstaw zrzut ekranu terminala, obrazujący całość procesu – aktywacja wirtualnego środowiska, uruchomienie i wykonanie skryptu, deaktywacja wirtualnego środowiska.

```
D:\ag1f57t.github.io\LAB_J\.venv\Scripts\python.exe D:\ag1f57t.github.io\LAB_J\hello.py
Hello Wojciech (57780). This environment is using Python version 3.13.2 at location D:\ag1f57t.github.io\LAB_J\.venv\Scripts\python.exe.

Process finished with exit code 0
```

Punkty:	0	1
---------	---	---

PORTOWANIE APLIKACJI Z LAB G/I

W dalszej części laboratorium przygotuj kompletną aplikację CRUD (Create, Read, Update, Delete) na podstawie LAB G/I. Wykorzystaj framework Flask w wersji 3.1.x.

Dokumentacja:

<https://flask.palletsprojects.com/en/stable/>

Wymagania:

- aplikacja musi się uruchamiać na porcie 57780;
- zastosowanie szablonów jinja2;
- ta sama baza danych SQLite; połączenie z wykorzystaniem modułu sqlite3; bez ORM (lub własny);
- zastosowanie szablonu layoutu i dziedziczących po nim szablonów poszczególnych akcji;
- wystarczy port CRUDA własnego modelu, nie trzeba przenosić encji Post;
- całość aplikacji (bez środowiska wirtualnego) należy zacommitować do repozytorium bezpośrednio w katalogu I:\PTW\lab-j;
- w projekcie zacommitowany musi być requirements.txt ze wszystkimi wymaganymi zależnościami.

W pozostałej części laboratorium, przenieś layout, style, migracje SQL (baza SQLite) oraz wszystkie operacje CRUD dotyczące Twojego modelu z aplikacji PHP do aplikacji we Flask (tylko Twój model, bez postów).

Wstaw zrzut ekranu kodu routingu i kontrolera listy:

```
@app.route('/')
def recipe_index():
    recipes = query_db('SELECT * FROM recipe')
    return render_template(template_name_or_list='recipe/index.html', recipes=recipes)
```

Wstaw zrzut ekranu kodu szablonu Jinja listy oraz szablonu Jinja layoutu:

```

1  {% extends 'layout.html' %}
2
3  {% block title %}Recipe List{% endblock %}
4  {% block body_class %}index{% endblock %}
5
6  {% block main %}
7      <h1>Recipes List</h1>
8
9      <a href="{{ url_for('recipe_create') }}">Create new</a>
10
11     <ul class="index-list">
12         {% for recipe in recipes %}
13             <li>
14                 <h3>{{ recipe['name'] }}</h3>
15                 <ul class="action-list">
16                     <li><a href="{{ url_for('recipe_show', recipe_id=recipe['id']) }}">Details</a></li>
17                     <li><a href="{{ url_for('recipe_edit', recipe_id=recipe['id']) }}">Edit</a></li>
18                 </ul>
19             </li>
20         {% endfor %}
21     </ul>
22 {% endblock %}
23

```

```

1  <!doctype html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, user-scalable=no, initial-scale=1.0, maximum-sc
6      <meta http-equiv="X-UA-Compatible" content="ie=edge">
7      <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
8      <title>{% block title %}Flask App{% endblock %}</title>
9  </head>
10 <body class="{% block body_class %}{% endblock %}">
11 <nav>
12     <ul>
13         <li><a href="{{ url_for('recipe_index') }}">Home</a></li>
14         <li><a href="{{ url_for('recipe_index') }}">Recipes</a></li>
15     </ul>
16 </nav>
17 <main>
18     {% block main %}{% endblock %}
19 </main>
20 <footer>{{ now }} Flask App</footer>
21 </body>
22 </html>
23

```

Wstaw zrzut ekranu listy (z danymi) w przeglądarce:

127.0.0.1:57780

⌂

🔗

🛡️

🚦

⚙️

📺

⏴

🎵

📺

📁

🔖

🛡️

🌐

☰

HOME RECIPES

Recipes List

Create new

beza edited

Details Edit

makaron

Details Edit

asd

Details Edit

©2026 Flask App

Punkty:	0	1
---------	---	---

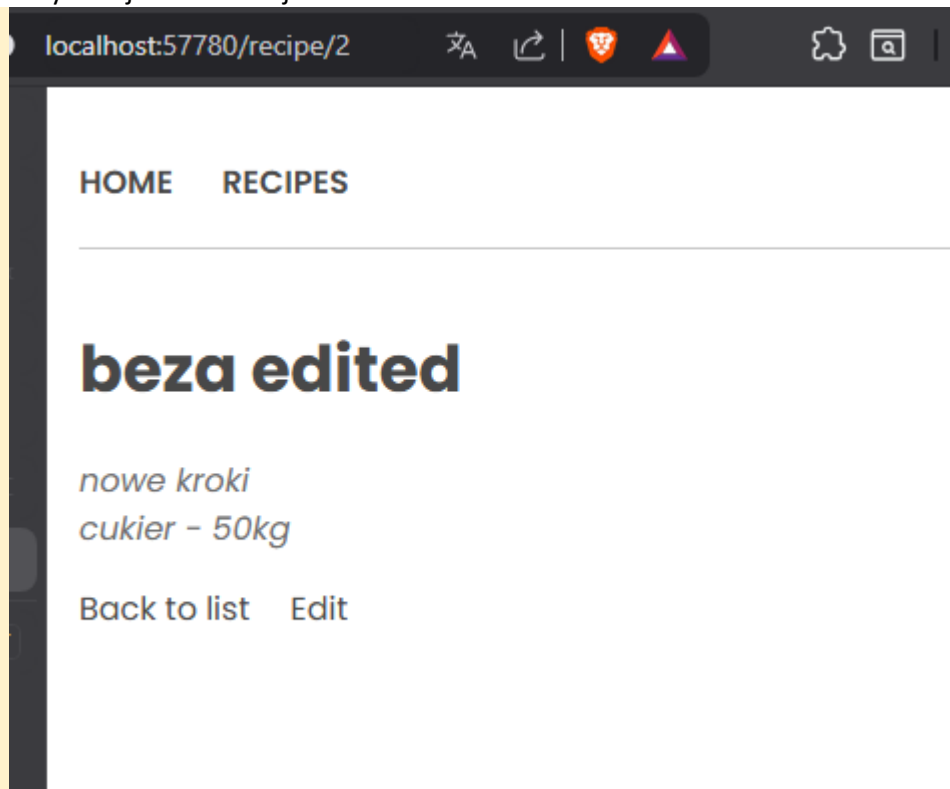
Wstaw zrzut ekranu kodu routingu i kontrolera podglądu pojedynczego modelu:

```
@app.route('/recipe/<int:recipe_id>')
def recipe_show(recipe_id):
    recipe = query_db(query='SELECT * FROM recipe WHERE id = ?', args=[recipe_id], one=True)
    if recipe is None:
        return render_template('404.html'), 404
    return render_template(template_name_or_list='recipe/show.html', recipe=recipe)
```

Wstaw zrzut ekranu kodu szablonu Jinja podglądu:

```
1 {% extends 'layout.html' %}
2
3 {% block title %}{{ recipe['name'] }} ({{ recipe['id'] }}){% endblock %}
4 {% block body_class %}show{% endblock %}
5
6 {% block main %}
7     <h1>{{ recipe['name'] }}</h1>
8
9     <article>
10         {{ recipe['steps'] }}
11     </article>
12     <article>
13         {{ recipe['ingredients'] }}
14     </article>
15
16     <ul class="action-list">
17         <li><a href="{{ url_for('recipe_index') }}">Back to list</a></li>
18         <li><a href="{{ url_for('recipe_edit', recipe_id=recipe['id']) }}">Edit</a></li>
19     </ul>
20 {% endblock %}
21
```

Wstaw zrzut ekranu podglądu w przeglądarce:



Punkty:

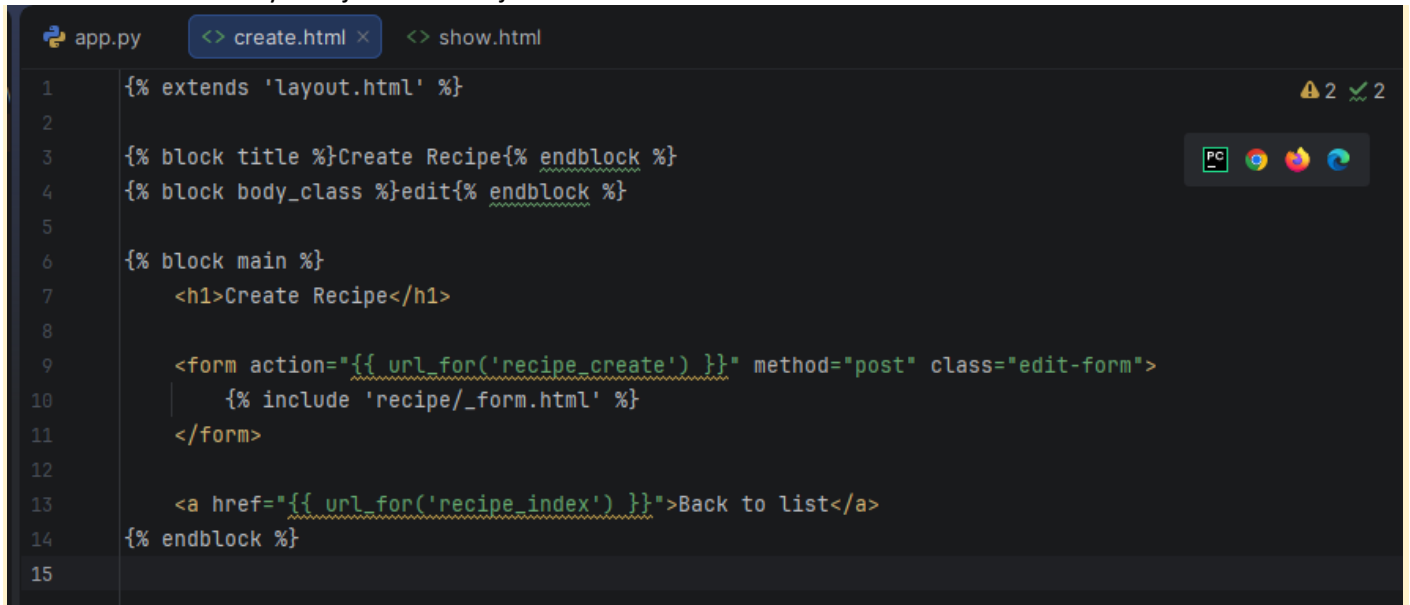
0

1

Wstaw zrzut ekranu kodu routingu i kontrolera tworzenia modelu:

```
@app.route(rule: '/recipe/create', methods=['GET', 'POST'])
def recipe_create():
    if request.method == 'POST':
        name = request.form['name']
        steps = request.form['steps']
        ingredients = request.form['ingredients']
        modify_db(
            query: 'INSERT INTO recipe (name, steps, ingredients) VALUES (?, ?, ?)',
            args: [name, steps, ingredients]
        )
        return redirect(url_for('recipe_index'))
    return render_template('recipe/create.html')
```

Wstaw zrzut ekranu kodu szablonu Jinja tworzenia:



The screenshot shows a code editor with a dark theme. The top bar has tabs for 'app.py', 'create.html' (active), and 'show.html'. The code is a Django template for 'create.html'. It extends 'layout.html' and defines a block 'main' containing an h1 'Create Recipe', a form for creating a recipe, and a link back to the list. The form uses the 'recipe_create' view and includes 'recipe/_form.html'. The link uses the 'recipe_index' view. The editor has a line number margin on the left (1-15) and a status bar on the right showing 2 warnings and 2 checks.

```
1 {% extends 'layout.html' %}
2
3 {% block title %}Create Recipe{% endblock %}
4 {% block body_class %}edit{% endblock %}
5
6 {% block main %}
7     <h1>Create Recipe</h1>
8
9     <form action="{{ url_for('recipe_create') }}" method="post" class="edit-form">
10         {% include 'recipe/_form.html' %}
11     </form>
12
13     <a href="{{ url_for('recipe_index') }}">Back to list</a>
14 {% endblock %}
15
```

Wstaw zrzut ekranu tworzenia w przeglądarce:

HOME

RECIPES

Create Recipe

Name

asd

Steps

asd

Ingredients

asd

Submit

Punkty:	0	1
---------	---	---

Wstaw zrzut ekranu kodu routingu i kontrolera(ów) edycji modelu:

```
67 @app.route(rule: '/recipe/<int:recipe_id>/edit', methods=['GET',
68     'POST'])
69     def recipe_edit(recipe_id):
70         recipe = query_db(query: 'SELECT * FROM recipe WHERE id = ?',
71             args: [recipe_id], one=True)
72         if recipe is None:
73             return render_template('404.html'), 404
74         if request.method == 'POST':
75             name = request.form['name']
76             steps = request.form['steps']
77             ingredients = request.form['ingredients']
78             modify_db(
79                 query: 'UPDATE recipe SET name = ?, steps = ?, ingredients
80                     = ? WHERE id = ?',
81                 args: [name, steps, ingredients, recipe_id]
82             )
83             return redirect(url_for('recipe_index'))
84         return render_template(template_name_or_list: 'recipe/edit.html',
85             recipe=recipe)
```

Wstaw zrzut ekranu kodu szablonu Jinja edycji:

```
1  {% extends 'layout.html' %}
2
3  {% block title %}Edit Recipe {{ recipe['name'] }} ({{ recipe['id'] }}){% endblock %}
4  {% block body_class %}edit{% endblock %}
5
6  {% block main %}
7      <h1>Edit Recipe {{ recipe['name'] }} ({{ recipe['id'] }})</h1>
8
9      <form action="{{ url_for('recipe_edit', recipe_id=recipe['id']) }}" method="post" class="edit-form">
10         {% include 'recipe/_form.html' %}
11     </form>
12
13     <ul class="action-list">
14         <li><a href="{{ url_for('recipe_index') }}">Back to list</a>
15         </li>
16         <li>
17             <form action="{{ url_for('recipe_delete', recipe_id=recipe['id']) }}" method="post">
18                 <input type="submit" value="Delete" onclick="return confirm('Are you sure?')">
19             </form>
20         </li>
21     </ul>
22 {% endblock %}
```

Wstaw zrzut ekranu edycji:

HOME

RECIPES

Edit Recipe beza edited (2)

Name

beza edited

Steps

nowe kroki

Ingredients

cukier - 50kg

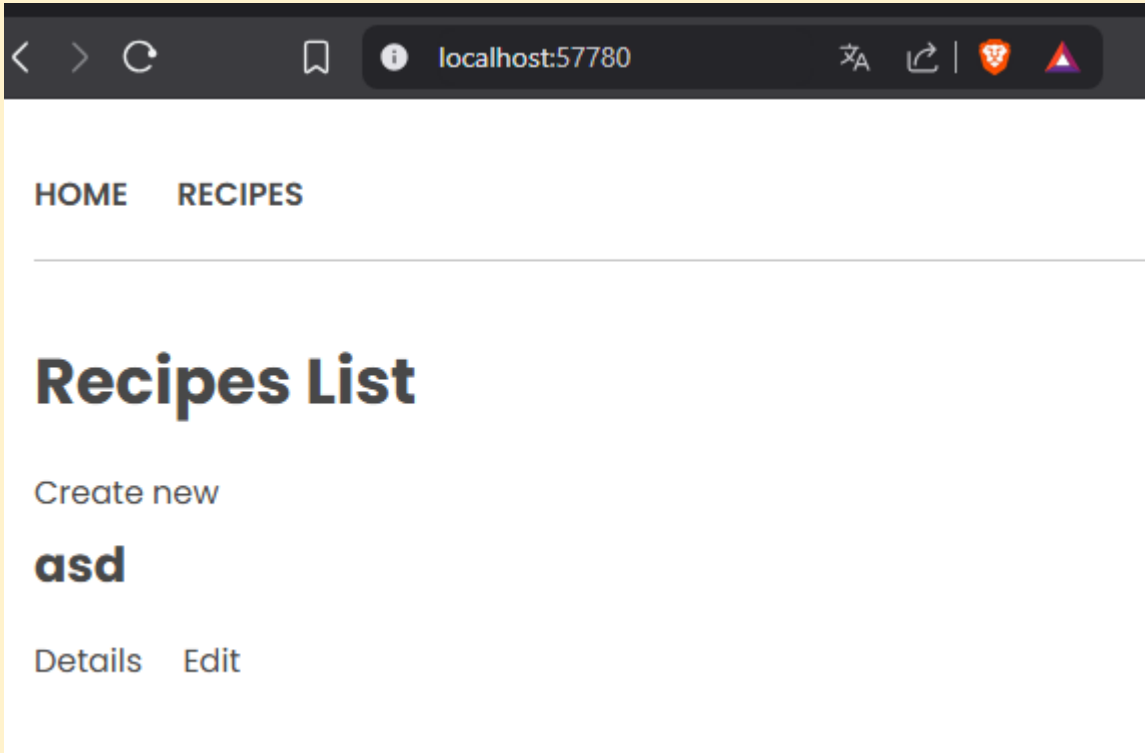
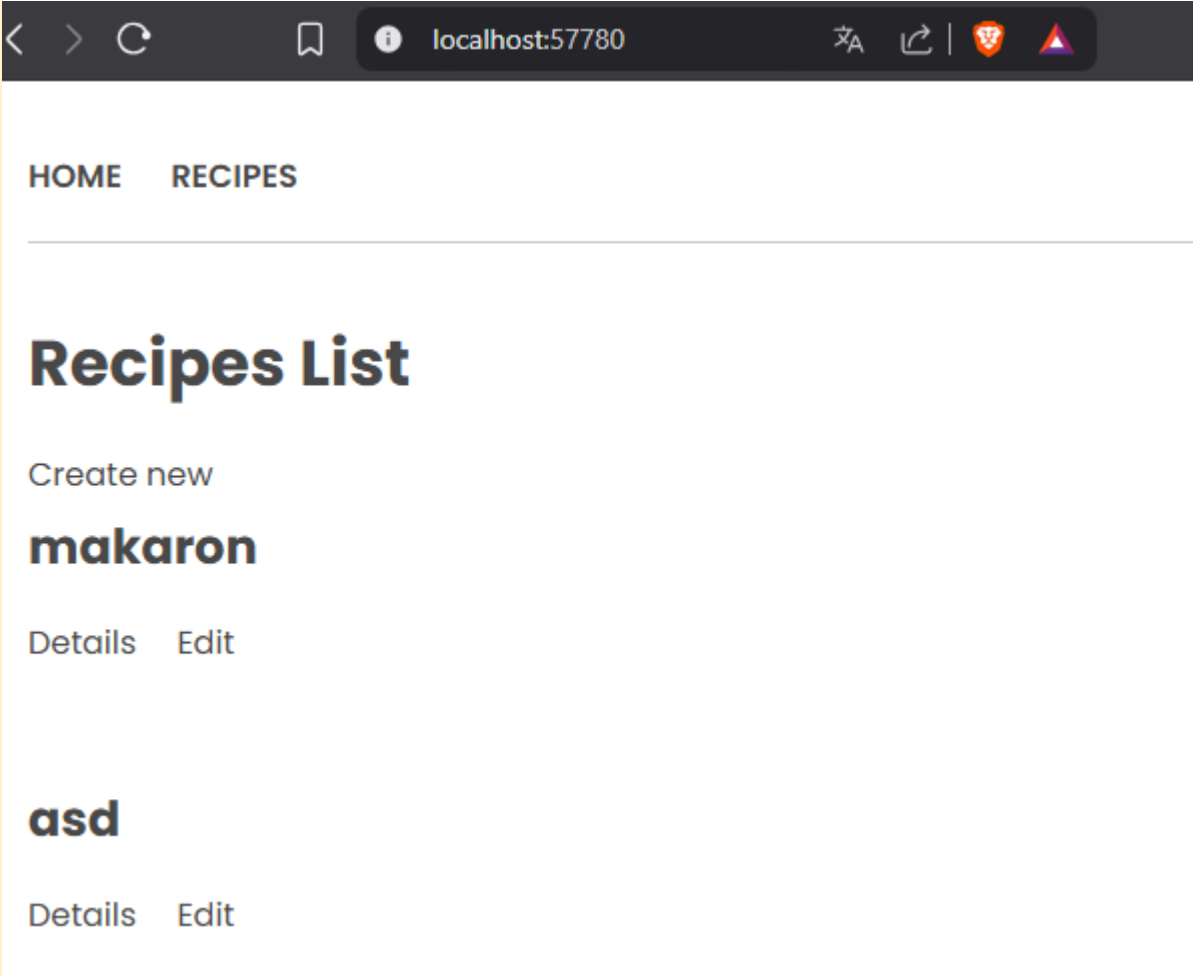
Submit

Punkty:	0	1
---------	---	---

Wstaw zrzut ekranu kodu routingu i kontrolera kasowania:

```
83 @app.route(rule: '/recipe/<int:recipe_id>/delete', methods=['POST'])
84 def recipe_delete(recipe_id):
85     recipe = query_db(query: 'SELECT * FROM recipe WHERE id = ?',
86                        args: [recipe_id], one=True)
87     if recipe is None:
88         return render_template('404.html'), 404
89     modify_db(query: 'DELETE FROM recipe WHERE id = ?',
90              args: [recipe_id])
91     return redirect(url_for('recipe_index'))
```

Wstaw zrzut ekranu działającego kasowania:



Punkty:	0	1
---------	---	---

COMMIT PROJEKTU DO GIT

Zacommituj i pushnij swoje rozwiązanie do swojego repozytorium GIT.

Upewnij się, czy wszystko dobrze się wysłało.

Podaj link do swojego repozytorium:

https://github.com/AG1F57T/ag1f57t.github.io/tree/main/LAB_J

PODSUMOWANIE

W kilku słowach/zdaniach napisz swoje przemyślenia odnośnie tego laboratorium. **Nie używaj LLM.** Który ze wszystkich poznanych dotychczas sposobów tworzenia aplikacji najbardziej przypadł Ci do gustu?

...podsumowanie...

Zweryfikuj kompletność sprawozdania. Utwórz PDF i wyślij w terminie.